Switching SSH from Password to Key Authentication - Windows 11

This guide walks you through switching your SSH connection from password authentication to key authentication. This is a one-time setup that makes connecting to your Linux server more secure and more convenient - no more typing a password every time you connect.

> Note: This guide covers Windows 11 as the local machine. Mac and Linux users have a slightly simpler process - both have the ssh-copy-id command built in natively, which replaces the longer PowerShell command used in Step 3. All other steps are broadly the same across platforms.

What is SSH key authentication and why use it?

SSH (Secure Shell) creates an encrypted connection between your local machine and a remote server. By default it uses password authentication, which works but has weaknesses:

- You have to type your password every time you connect
- Passwords can be guessed or brute-forced by automated attacks
- If your password is compromised, your server is at risk

Key authentication replaces the password with a cryptographic key pair:

- A private key - stored on your local machine only, never shared with anyone
- A public key - copied to the server, like a padlock that only your private key can open

The result: connecting to your server becomes both more secure and simpler. No password required - just type ssh and you're in.

What you need before starting

- A Windows 11 PC (the local machine you connect from)
- A Linux server running Ubuntu (the remote server you connect to)
- Your server's IP address (either local network IP or Tailscale IP)
- Your server username and current password (needed one last time during setup)
- PowerShell - use this throughout the entire guide, not Command Prompt

The guide

Step 1 - Generate your SSH key pair

Open PowerShell on your Windows PC and run:

```
ssh-keygen -t ed25519 -C "your@email.com"
```

Replace the email address with your own. This command generates two files:

- id_ed25519 - your private key. Never share this with anyone.
- id_ed25519.pub - your public key. This is what goes to the server.

Breaking down the command:

- -t ed25519 - the type of key to generate. ed25519 is the modern standard - more secure than older alternatives (like rsa), faster, and uses a shorter key length
- -C "your@email.com" - a comment added to the key to help identify it. It does not affect how the key works

When prompted:

- Where to save the key - press Enter to accept the default location
- Passphrase - press Enter for none (or type one for an extra layer of security)
- Confirm passphrase - press Enter again

> Note: The default save location is C:\Users\yourusername\.ssh\ - you will find both key files there after this step.

Step 2 - Start the SSH agent and add your private key

The SSH agent is a background service that holds your private key in memory so you do not have to specify it manually every time you connect.

Check if the agent is running:

```
Get-Service ssh-agent
```

If it shows as Stopped, run these two commands to start it and set it to start automatically:

```
Get-Service ssh-agent | Set-Service -StartupType Automatic
Start-Service ssh-agent
```

Now add your private key to the agent:

```
ssh-add ~/.ssh/id_ed25519
```

If you set a passphrase in Step 1, you will be asked for it here. After this, the agent holds your key for the rest of your Windows session.

Step 3 - Copy your public key to the server

Linux has a built-in command called ssh-copy-id for this, but Windows does not. Instead, use this PowerShell command:

```
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh username@server-ip "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
```

Replace username with your server username and server-ip with your server's IP address.

This will ask for your server password one last time. After this step you will not need it again.

What this command does, step by step:

- type $env:USERPROFILE\.ssh\id_ed25519.pub - reads your public key file on Windows
- | ssh username@server-ip - pipes (sends) it to your server over SSH
- mkdir -p ~/.ssh - creates the .ssh folder on the server if it does not already exist
- chmod 700 ~/.ssh - sets the correct permissions on the folder (only you can access it)
- cat >> ~/.ssh/authorized_keys - appends your public key to the server's authorized keys file
- chmod 600 ~/.ssh/authorized_keys - sets the correct permissions on the authorized keys file (only you can read and write it)

Step 4 - Test key authentication

Before making any further changes, confirm that key authentication is actually working:

```
ssh username@server-ip
```

- If it connects without asking for a password - key auth is working. Continue to Step 5.
- If it still asks for a password - do not proceed. Go back and check Step 3.

> **Important: Once key auth is confirmed working, open two more SSH sessions to your server and keep all three open. These act as safety nets for the steps ahead - if something goes wrong you will still have active sessions to fix it.**

Step 5 - Verify the public key and permissions on the server

In one of your open SSH sessions, check the public key copied correctly:

```
cat ~/.ssh/authorized_keys
```

You should see your public key, starting with:

```
ssh-ed25519 AAAAC3... your@email.com
```

Check the permissions are correct:

```
ls -la ~/.ssh/
```

You should see:

```
drwx------  .ssh/            (700) -rw-------  authorized_keys  (600)
```

If the permissions are wrong, fix them:

```
chmod 700 ~/.ssh chmod 600 ~/.ssh/authorized_keys
```

> Why permissions matter: SSH is strict about security. If the permissions on .ssh or authorized_keys are too open, SSH will refuse to use the key and fall back to password authentication. The permissions must be exactly: .ssh folder = 700 (only you can access it), authorized_keys = 600 (only you can read and write it).

Step 6 - Edit the SSH server configuration

In one of your open SSH sessions, edit the SSH server config file:

```
sudo vim /etc/ssh/sshd_config
```

Use vim's search to find each setting quickly by typing / followed by the setting name:

```
/PubkeyAuthentication /AuthorizedKeysFile /PasswordAuthentication
```

Find and set these three lines exactly as shown. Remove the # from the start if present, and make sure there is only one of each line (no duplicates):

```
PubkeyAuthentication yes AuthorizedKeysFile .ssh/authorized_keys
PasswordAuthentication no
```

Save and exit:

```
Esc :wq
```

Step 7 - Restart the SSH service

In Session 1, restart the SSH service:

```
sudo systemctl restart ssh
```

Immediately switch to Session 2 and try running any command:

```
pwd
```

- If Session 2 still responds - everything worked. Continue to Step 8.
- If Session 2 dropped - work through the recovery steps below in order, and don't panic.

Recovery if you get locked out

- Try reconnecting: ssh username@server-ip
- If that fails, try your Tailscale address if you have one set up: ssh username@tailscale-ip
- If still locked out, you will need physical access to the server (keyboard and monitor connected directly)

To re-enable password authentication via physical access:

```
sudo vim /etc/ssh/sshd_config
```

Change PasswordAuthentication back to yes, then:

```
sudo systemctl restart ssh
```

You should now be able to SSH in with your password again. Go back to Step 5 and check what went wrong before trying again.

Step 8 - Create an SSH config file on Windows

This optional but highly recommended step lets you connect to your server by name rather than typing the full IP address every time.

On your Windows PC, create a new file at exactly this location:

```
C:\Users\yourusername\.ssh\config
```

> **Important: the file must be named config with no file extension. Not config.txt.**

Add this content, replacing the values with your own:

```
Host myserver     HostName server-ip     User username     IdentityFile ~/.ssh/id_ed25519
```

Replace myserver with whatever name you want to use (e.g. the hostname of your server), server-ip with your server's IP address, and username with your server username.

To save the file without Notepad adding a .txt extension:

- Open Notepad
- File > Save As
- Change the "Save as type" dropdown to All Files
- Type the filename as exactly config (no extension)
- Save it in C:\Users\yourusername\.ssh\

> Tip: If you use Visual Studio Code or Notepad++, these editors make it easier to save files without unwanted extensions. Simply save the file as config with no extension.

Verify it saved correctly in PowerShell:

```
Get-ChildItem C:\Users\yourusername\.ssh\
```

You should see three files: config, id_ed25519, id_ed25519.pub - no .txt extension on config.

Step 9 - Test the final easy login

Open a fresh PowerShell window - but keep your existing SSH sessions open for now as a safety net. Connect using just the name you set in the config file:

```
ssh myserver
```

It should connect instantly with no password. This works because PowerShell reads your config file and automatically uses the correct IP address, username, and private key.

If it connects successfully, you can now close your existing safety net sessions. Setup is complete.

If it does not connect, do not close your existing sessions - you will need them to fix the problem. Run the verbose flag to diagnose what is going wrong:

```
ssh -v myserver
```

The -v (verbose) flag shows a detailed step-by-step log of exactly what SSH is attempting and where it is failing. For example it might show the wrong key file path, a permission issue, or a connection refused error - each of which points you back to a specific earlier step to fix.

Step 10 - Final verification

Once connected, confirm password authentication is fully disabled:

```
sudo grep PasswordAuthentication /etc/ssh/sshd_config
```

Should return:

```
PasswordAuthentication no
```

Confirm public key authentication is enabled:

```
sudo grep PubkeyAuthentication /etc/ssh/sshd_config
```

Should return:

```
PubkeyAuthentication yes
```

Golden rules going forward

- Never delete your private key (id_ed25519) from your Windows PC - it is the only thing that can open your server. If you lose it you will need physical access to re-enable password authentication and start over.
- Never share your private key with anyone, ever. It is the equivalent of handing someone the keys to your server.
- Never commit your private key to GitHub or any other public location. Only the .pub file is safe to share publicly, though you rarely need to.
- If you get a new Windows PC or reinstall Windows, you will need to generate a new key pair and repeat Steps 3-7 to copy the new public key to your server.

- Consider backing up your private key to a secure location (an encrypted USB drive or password manager) so you are not locked out if your Windows PC fails.
- If you ever suspect your private key has been compromised, remove it from the server immediately by deleting its entry from ~/.ssh/authorized_keys on the server, then generate a new key pair.

Last updated June 2026.